



Knowledge Refresh

Module 1

Objectives

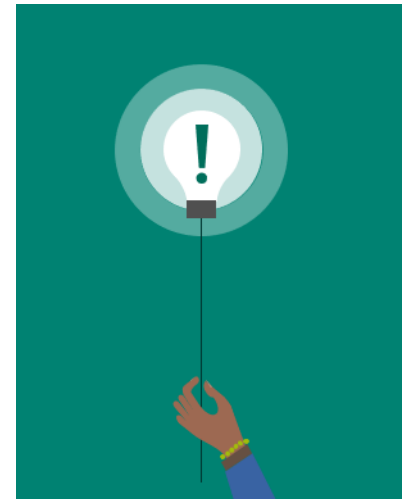
The purpose of this learning unit is to do a quick review of expected knowledge:

PowerShell Foundations Course

- Script Basics
- Flow Control
- Functions

PowerShell IT Management Course

- Object Models
- Regular Expressions



Script Basics

What should you consider when building / working with scripts?

Modules

Execution policies

Dependencies

Synopsis

Script signing

Flow Control

The loops in PowerShell repeat code

Five different logical variations:

Name	Loop Type	Features
While	Conditional	Tests for \$True condition
Do While	Conditional	Tests for \$True condition Code block runs at least once
Do Until	Conditional	Tests for \$False condition Code block runs at least once
For	Conditional	Tests for \$True condition Includes Initialization and Repeat blocks
Foreach	Enumeration	Runs code once for each item in collection/array Choose automatic variable name

Logical tests

The tests alter code flow based on the outcome of the test

Five different logical variations:

Name	Features
If ...	Tests condition and run code block on \$true
If ... else ...	Tests condition and run code block on \$true, run else code block on \$false
If ... elseif ...	Tests condition and run code block on \$true, on \$false, Tests 2 nd condition and run code block on \$true
If ... elseif ... else	Tests condition and run code block on \$true, on \$false, Tests 2 nd condition and run code block on \$true on \$false, run else code block
switch	Test condition and run <u>All</u> code blocks that match test

Flow Control Keywords

Keyword	Description	Example(s)
Break	Immediately exit Loop Example breaks after 1 match to avoid multiple matches	<pre>Switch -wildcard ("WMF 5.0") { "WMF 5.0" {"Matched First"; Break} "W*" {"Matched Second"} }</pre> Matched First
Continue	Immediately returns to top of loop Example skips over 2	<pre>\$c = 0 while (\$c -lt 3) { \$c++ if (\$c -eq 2) {Continue} Write-Host \$c }</pre> 1 3

Flow Control Keywords (Continued)

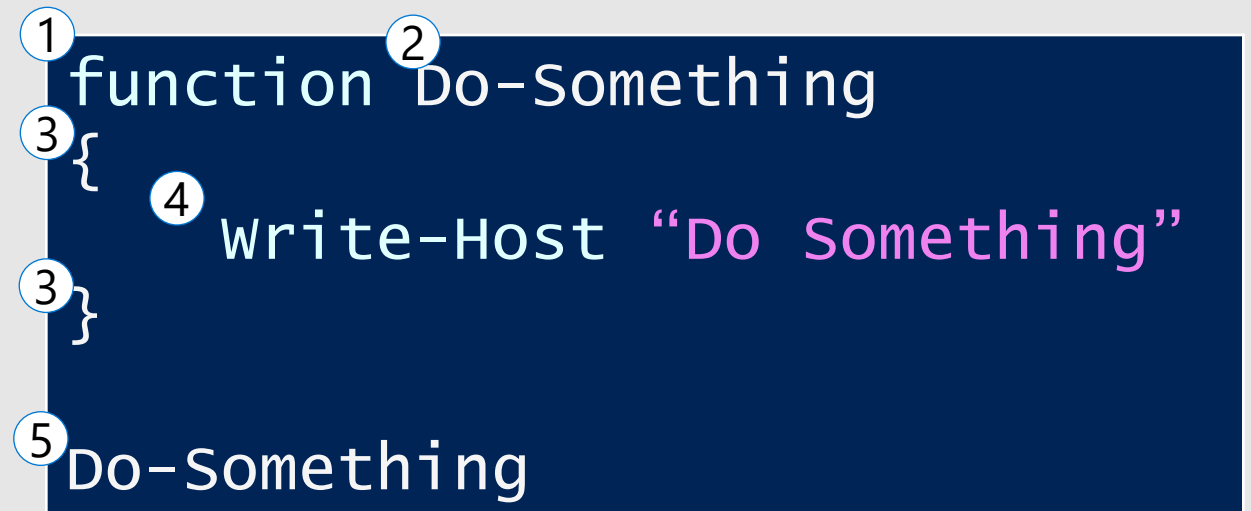
Keyword	Description	Example(s)
Return	Exits current 'scope', which can be a function, script, or script block Note: Return can appear alone or followed by a value or expression	<pre>function Test-Return (\$val) { if (\$val -ge 5) {return \$val} Write-Host "Reached end of function" }</pre> <pre>PS C:\> Test-Return 1 Reached end of function PS C:\> Test-Return 6 6</pre>
Exit	Exit current script or session – Optional ErrorLevel Numeric Code	<pre>PS C:\> Exit 10</pre>

What is a Function?

- Named Script Block
- Reusable block of PowerShell code
- Reduces size of code and increases reliability
- Can accept parameter values and return output
- Advanced Functions behave like Cmdlets
- Can be created with help topics that can be used with Get-Help (like cmdlets)

What Does a Function Look Like?

1. Function Keyword
2. Function Name
3. Matching Open/Close Curly Braces
4. Statement List
5. Use Function



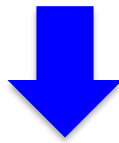
```
1 function 2 Do-Something
3 {
4     Write-Host "Do Something"
3 }
5 Do-Something
```

The diagram illustrates the components of a function definition and its usage. It features a dark blue rectangular box containing white text. The text is annotated with numbered circles (1-5) corresponding to the list on the left. The function definition is shown on three lines: the first line contains the keyword 'function' (1) and the name 'Do-Something' (2); the second line contains an opening curly brace '{' (3); the third line contains a closing curly brace '}' (3). The statement list is shown on the fourth line, indented, containing the command 'Write-Host' (4) followed by the string 'Do Something' in pink. Below the function definition, on a separate line, is the function call 'Do-Something' (5).

Parameters in a function

- Defined using param statement
- Used just like variables
- Passed in using Dash notation
- Can have advanced attributes

```
function Get-ServiceInfo
{
    Param ($svc, $computer)
    Get-Service -Name $svc -RequiredServices -ComputerName $computer
}
```



```
PS C:\> Get-ServiceInfo -svc spooler -computer localhost
```

Object-Based Shell Vocabulary

Objects are the way we represent data in PowerShell

- Everything in PowerShell is represented as an **OBJECT**
- The Object consist of information (**PROPERTIES**) and actions (**METHODS**)
- PROPERTIES and METHODS are known as **MEMBERS**
 - Explore them with Get-Member cmdlet
- A **TYPE** represents the template of available MEMBERS
- A specific object is an **INSTANCE** of a TYPE

```
$service = get-service alg
```

```
> $service.DisplayName  
Application Layer Gateway Service  
> $service.Stop()
```

Dot-Notation to
access member

Methods use parens,
which sometimes
take data

Regular Expression (RegEx) overview

RegEx - Think of it as “Wild cards on steroids”

Why is it worth learning?

- Data validation
- Data extraction
- Data manipulation

What gives it the reputation for being difficult?

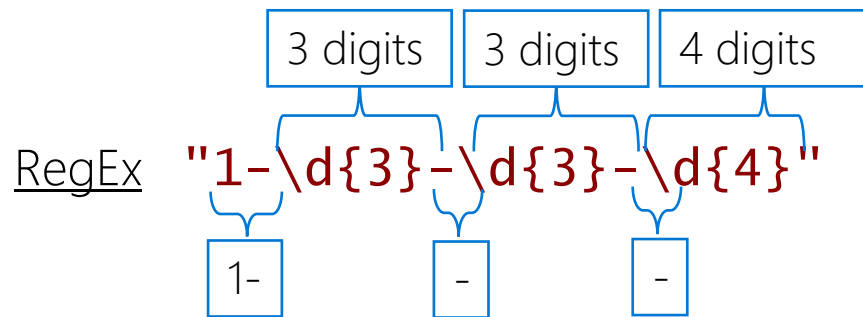
- Symbols will look different than ‘normal’ PowerShell
- RegEx pattern is difficult to read

RegEx examples

The Data:

1. Michele Holden	English (US)	1-190-555-1087
2. Mildred Frye	English (US)	1-586-555-1081
3. Cleveland Matheson	English (US)	1-560-555-1014
4. Scotty Frazer	English (US)	1-845-555-1053
5. Mollie Walter	English (US)	1-701-555-1097

Need: Only phone numbers



How we use it:

By default `-Match` pulls out the first thing it sees into `$matches[0]`.

The other matches get ignored for now.

The Code:

```
$file = get-content C:\users.txt
$regex = "1-\d{3}-\d{3}-\d{4}"
foreach ($line in $file)
{
    if($line -match $regex)
    {
        $matches[0]
    }
}
```

The Output:

```
1-190-555-1087
1-586-555-1081
1-560-555-1014
1-845-555-1053
1-701-555-1097
```

Example: split text with `-split` or `-csplit` using RegEx

```
PS C:\> $a = @"
1The first line.
2The second line.
3The third of three lines.
"@
PS C:\> $a -split "\d" # Any digit

The first line.

The second line.

The third of three lines.
```

Questions?



